

Motion Optimization of Wheeled Soccer Robot Using Kalman Filter on YOLOv8-Based Object Tracking

Motion Optimization of Wheeled Soccer Robot Using Kalman Filter on YOLOv8-Based Object Tracking

Fikri Rivandi¹, Feri Candra²

^{1,2}Department of Informatics Engineering, Faculty of Engineering, Universitas Riau, Pekanbaru, Indonesia
fikri.rivandi2583@student.unri.ac.id, fer_i@eng.unri.ac.id

ABSTRACT

The Indonesian Wheeled Soccer Robot Contest (KRSBI-Beroda) requires robots to perform real-time ball detection and tracking in dynamic environments. Single-frame detection-based tracking methods such as YOLOv8 often face hardware limitations and visual disturbances like occlusion, resulting in unstable robot motion. This study proposes an optimization of the KRSBI-Beroda robot vision system by integrating the You Only Look Once version 8 (YOLOv8) algorithm as an object detector and the Kalman Filter as a predictive position estimator. A frame-skipping approach is implemented to reduce computational load on the ASUS K401U laptop, where YOLOv8 is executed periodically with an interval of 3 frames, while the Kalman Filter predicts the ball's position between detection frames. Experimental results show a significant increase in frame rate (FPS) from 13.6 to 38.77 (an increase of 185%). The optimized system also eliminates tracking loss (0 occurrences) in occlusion scenarios and reduces mission completion time by 62% (from 15.17 seconds to 5.76 seconds). This integration has proven effective in stabilizing robot motion, reducing motion command jitter, and enhancing system reliability in soccer robot competitions.

Keywords : KRSBI-Beroda, YOLOv8, Kalman Filter, Object Tracking, Frame Skipping, Soccer Robot, Computer Vision.

Submitted:

Article Info :
Reviewed:

Accepted :

Copyright © 2022 – CSRID Journal. All rights reserved.

1. INTRODUCTION

The advancement of computer vision and deep learning technologies has driven rapid progress in autonomous robotics, particularly in the Indonesian Robot Contest (KRI), specifically the Indonesian Wheeled Soccer Robot Contest (KRSBI-Beroda) category [1]. In this competition, robots are required to detect and track ball movements in real-time to perform dribbling maneuvers and score goals. Failure in ball tracking, whether due to computational limitations or occlusion disturbances, can be fatal to the robot's performance on the field [2].

In the early stages of KRSBI-Beroda development, many teams relied on the HSV color filtering method for ball detection. This method is computationally lightweight but highly sensitive to changes in light intensity, making it unreliable in dynamic field conditions [2]. This weakness becomes critical because KRSBI-Beroda matches often take place under varying and uncontrolled lighting conditions [1].

With technological advancements, Convolutional Neural Network (CNN)-based methods such as You Only Look Once (YOLO) have been widely adopted by KRSBI teams to improve detection accuracy [3]. YOLO can detect objects with high precision even under varying lighting conditions [4]. The YOLOv8 architecture used in this study is the latest version with improvements in model efficiency, feature extraction, and computational speed [5, 6]. Research by Farhan & Candra [3] demonstrated that

the YOLOv8 model achieves an accuracy of 95.87% for ball and goal detection on KRSBI-Beroda robots.



Figure 1. Illustration of KRSBI-Beroda Match Environment
(Source: kontesrobotindonesia.id)

However, implementing YOLOv8 on limited hardware, a common constraint among university robotics teams using consumer-grade laptops, introduces new problems: high computational load causes low frame rate (FPS) and lag [7]. As a result, the robot often loses track of the ball when it moves quickly or when the ball is partially occluded [8]. Occlusion is a major cause of increased false-negative detection rates, which can degrade overall tracking performance [9]. This issue is universal across KRSBI participants who face similar budget and hardware constraints.

To address these issues, this study proposes the integration of YOLOv8 with the Kalman Filter and a frame-skipping strategy. The Kalman Filter is a lightweight recursive estimation algorithm capable of predicting the state of a dynamic system based on previous observation data [10]. Saputra [11] proved that Kalman Filter integration on KRSBI-Beroda robots can predict ball movement with an average measurement error of 1.06 for the x coordinate and 7.34 for the y coordinate. Meanwhile, the frame-skipping strategy has been proven effective in increasing processing speed by up to 1.5 times without significant accuracy degradation [12].

The proposed method offers a practical and cost-effective solution for KRSBI teams seeking to enhance their robot's vision performance without upgrading expensive hardware. By shifting part of the computational burden from the neural network to a lightweight mathematical estimator, the system maintains real-time performance while preserving detection accuracy and tracking continuity.

2. METHOD

A. Research Design and Model Preparation

This study employs a quantitative experimental approach by comparing two systems: a Baseline System (pure YOLOv8 inference performed on every frame) and an Optimized System (YOLOv8 integrated with the Kalman Filter and a frame-skipping strategy). The experiments were conducted on a representative KRSBI-Beroda robot platform equipped with an ASUS K401U laptop and an omnidirectional camera system.

Prior to integration, the YOLOv8 model was pre-trained using a specialized dataset captured directly from the robot's omnidirectional camera. This ensures the model's resilience against the unique radial distortion and varying lighting conditions of the field. The output of this detection model consists of bounding box parameters (x_c, y_c, w, h) along with a confidence score. In the optimized architecture, these exact parameters serve directly as the noisy measurement vector (Z_k) input for the Kalman Filter's correction stage.

B. Optimized System Architecture and Frame Skipping

In the optimized system, the tracking architecture is designed to reduce computational load without sacrificing tracking continuity. The system applies frame skipping with an interval of $SL = 3$, meaning YOLOv8 inference is performed on the 1st frame, while the 2nd and 3rd frames rely entirely on predictions from the Kalman Filter [13].

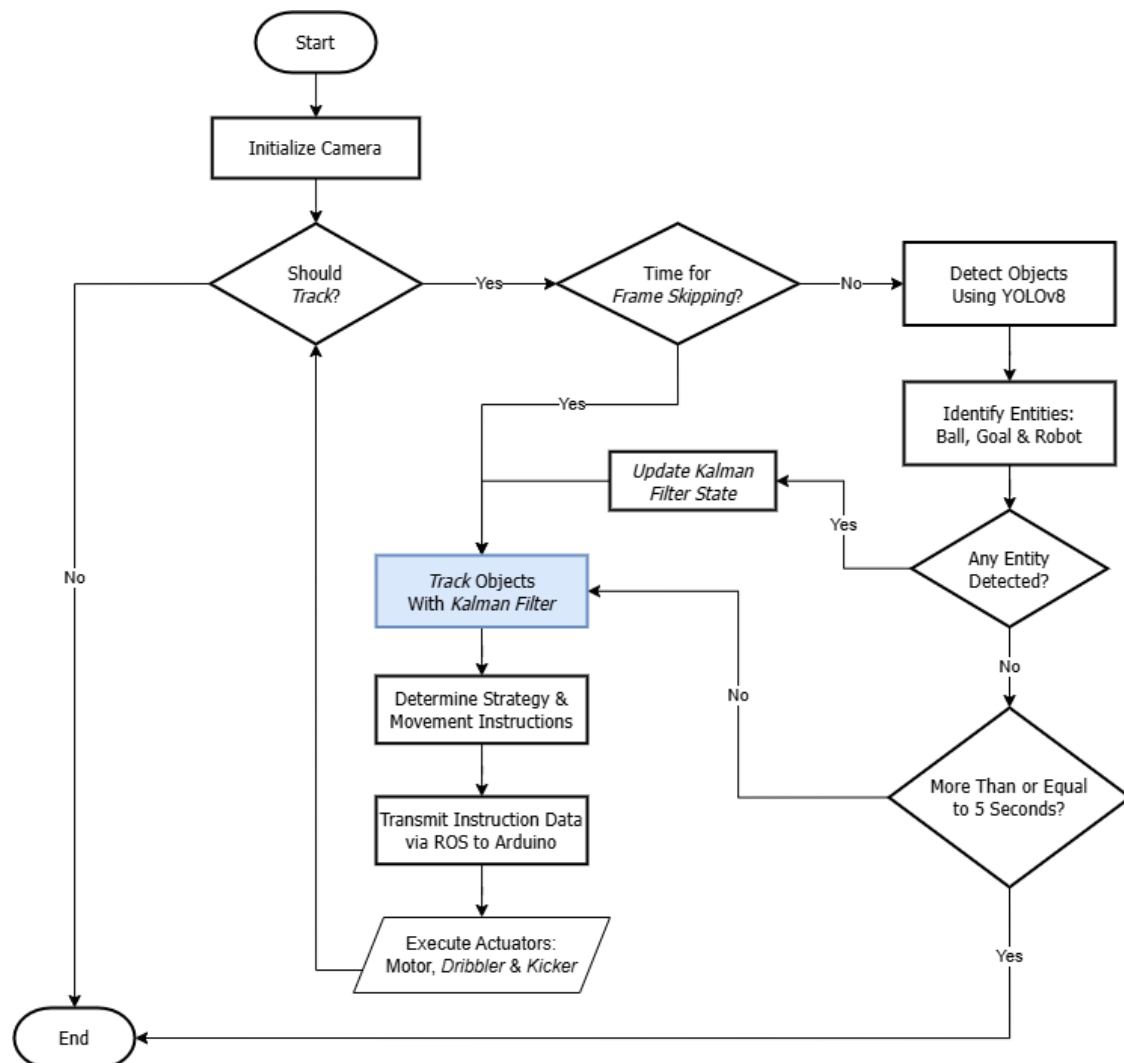


Figure 2. Workflow of the Optimized Tracking System with Frame Skipping and Kalman Filter

The workflow illustrated in Figure 2 demonstrates the adaptive tracking mechanism. To handle object occlusion, the system incorporates a loss-tolerance threshold. If YOLO fails to detect the ball on a scheduled frame (due to visual obstruction), the Kalman Filter bypasses the correction phase and purely

executes the prediction phase using its last known velocity vector. The track is only discarded if the object remains undetected for a continuous duration of 5 seconds, providing robust continuity during momentary occlusions.

The state vector X is defined as:

$$X = [x, y, v_x, v_y, w, h]^T \quad (1)$$

where (x, y) is the center coordinate of the ball, (v_x, v_y) is the object's velocity, and (w, h) are the dimensions of the bounding box. The motion model used is Constant Velocity with the transition matrix F :

$$F = \begin{bmatrix} 1 & 0 & dt & 0 & 0 & 0 \\ 0 & 1 & 0 & dt & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix} \quad (2)$$

The observation matrix H only reads (x, y, w, h) because YOLO does not directly provide velocity data:

$$H = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix} \quad (3)$$

The Kalman Filter process consists of two main stages [6]:

1. **Prediction Stage:** Predicts the current state based on the previous state.
2. **Correction Stage:** Updates the prediction with new measurements from YOLO.

C. Software Implementation

The system is implemented using:

- **ROS (Robot Operating System)** [14]: For inter-node communication and robot system management.
- **OpenCV v3.4** [15]: For image processing and visualization
- **NumPy v2.3** [16]: For Kalman Filter matrix computation.
- **Ultralytics YOLOv8** [6]: For object detection inference.

D. Vision-Based Motion Control and Grid Localization

A critical component of the methodology is translating the continuous pixel coordinates (x, y) generated by the tracking system into discrete mechanical commands for the robot's actuators. Because the robot utilizes an omnidirectional camera that captures a 360° distorted field of view, the system implements a Grid Localization algorithm to map the spatial environment accurately. In this approach, the visual frame is logically divided into an $(n \times m)$ grid matrix through an initial calibration process.

Once the Kalman Filter outputs the predicted coordinates of the ball, the algorithm maps these continuous coordinates to a specific grid zone. Each zone is pre-assigned a corresponding navigational instruction. For instance, if the ball is located in the top-center zone of the frame, the system issues a "forward" command to approach the target. If the ball falls within the left or right zones, it issues "turnLeft" or "turnRight" commands to adjust the robot's heading. Furthermore, if no object is tracked within the frame, a "search" rotation is automatically initiated. These string-based commands are continuously published via the *rosserial* communication protocol. The laptop, acting as the high-level processor, transmits these messages to the Arduino Mega 2560 microcontroller, which subsequently

translates the strings into precise Pulse Width Modulation (PWM) signals to drive the omnidirectional wheels.

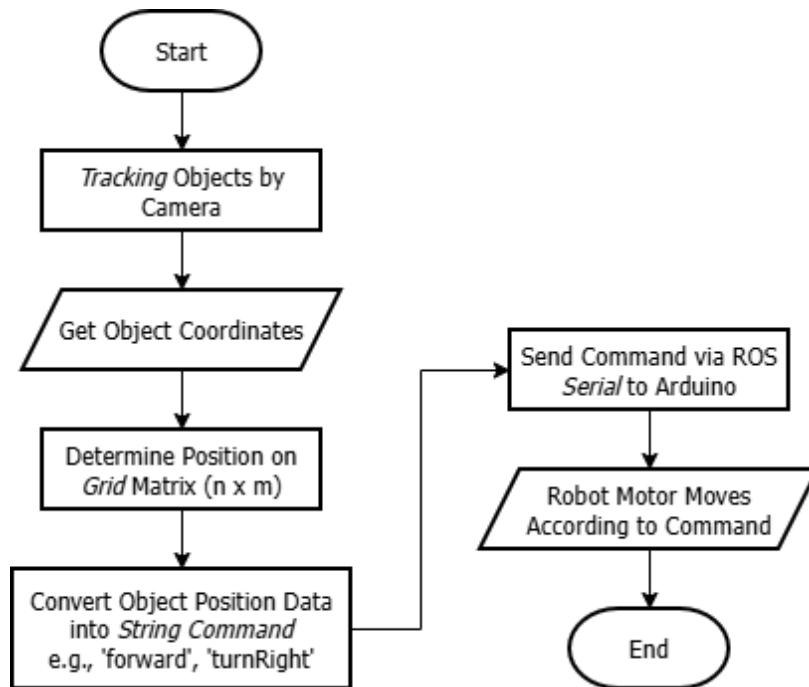


Figure 3. Flowchart of the Grid Localization Algorithm for Motion Control

E. Evaluation Scenarios



Figure 4. Visual outcome of the static test on the baseline system showing a low frame rate (avg. 13.60 FPS)

Testing was conducted in the ERC UNRI laboratory, utilizing a green carpet setup with physical obstacles to simulate match conditions. The evaluation is categorized into three main scenarios:

1. **Static Test:** Measures the computational efficiency by recording the average Frames Per Second (FPS) over a 30-second duration (10 repetitions) when observing a stationary ball.
2. **Occlusion Test:** Evaluates the robustness of the spatial memory. Metrics include the Tracking Loss Count (TLC) and Total Loss Duration (TLD) in seconds. Scenarios involve Mild Occlusion ($\pm 30\%$), Moderate Occlusion ($\pm 60\%$), and Momentary Occlusion (1-2 seconds fully covered behind an obstacle).
3. **Dynamic Test:** Simulates a full goal-scoring mission. The performance is quantified by measuring the total mission completion time, the Goal Rate percentage, and mechanical stability, which is calculated by observing the frequency of jitter (rapid, erratic switching of motion commands).

3. RESULT AND DISCUSSION

A. Computational Performance Analysis

Figure 4 and Figure 5 provide a direct visual comparison of the robot's vision interface during the static test. As depicted in Figure 3, forcing the YOLOv8 model to perform full deep learning inference on every single frame creates a severe computational bottleneck on the ASUS K401U laptop, limiting the processing speed to an average of only 13.60 FPS (detailed in Table 1).



Figure 4. Visual outcome of the static test on the baseline system showing a low frame rate (avg. 13.60 FPS)



Figure 5. Visual outcome of the static test showing stable detection and enhanced FPS (avg. 38.77) in the optimized system

As depicted in Figure 5, the graphical user interface of the robot's vision system demonstrates the real-time tracking capabilities of the optimized system. During the static test, the system consistently maintained a high frame rate, averaging around 38.77 FPS on the interface. This visual evidence highlights the efficiency of the frame-skipping strategy, which successfully prevents computational bottlenecks without losing track of the stationary ball.

Based on the static test results summarized in Table 1, the optimized system successfully increased the average FPS significantly.

Table 1. Comparison of Average FPS in Static Test

Repetition	Average FPS	
	Baseline System	Optimized System
1	13.91	41.28
2	14.34	36.29
3	13.54	37.61
4	13.73	43.16
5	13.52	34.38
6	13.48	35.30

Repetition	Average FPS	
	Baseline System	Optimized System
7	12.68	40.05
8	12.52	38.83
9	15.09	44.00
10	13.21	36.81
Average	13.60	38.77

The 185% increase in FPS occurs because the Kalman Filter is significantly more computationally lightweight ($O(n^2)$ simple matrix operations) compared to the YOLOv8 neural network inference, which intensively consumes CPU/GPU resources. With an average FPS of 38.77, the system meets the real-time requirement (>30 FPS), and the robot can react more quickly to dynamic ball movements. This result is consistent with the research by Taylor et al. [13], which proved that Kalman Filter integration significantly reduces processing time.

B. Occlusion Resilience

Figure 6 and Figure 7 illustrate the direct comparison between the two systems under a moderate 60% occlusion scenario. As observed in Figure 6, the baseline system completely fails to draw a bounding box around the partially hidden ball, accumulating a Tracking Loss Count (TLC) of 96. Conversely, Figure 7 demonstrates the optimized system's robustness; it successfully retains the bounding box (indicated by the [kf] label) by relying on the Kalman Filter's ongoing estimation, keeping the TLC at a perfect 0.

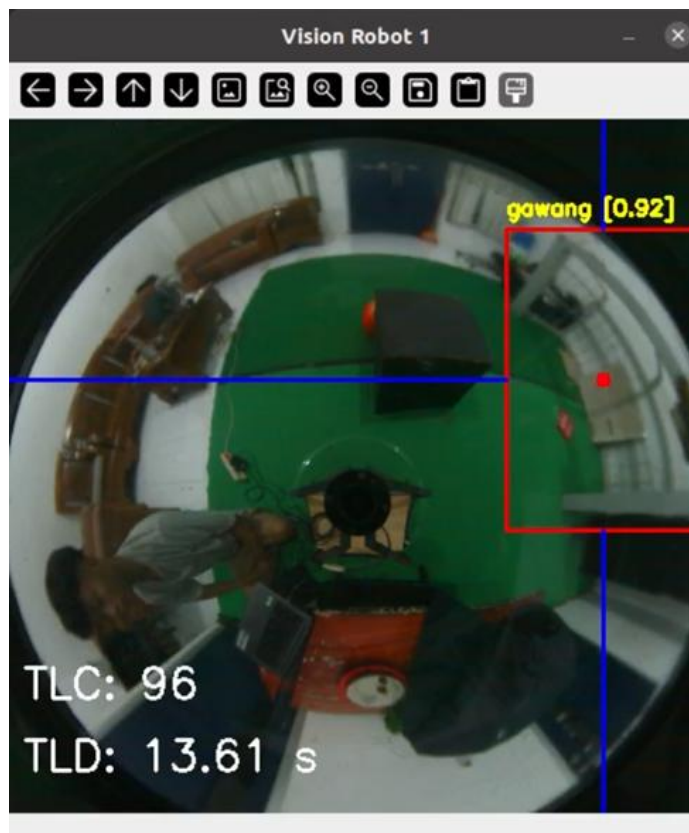


Figure 6. Baseline system failing to detect ball under 60% occlusion (TLC: 96)

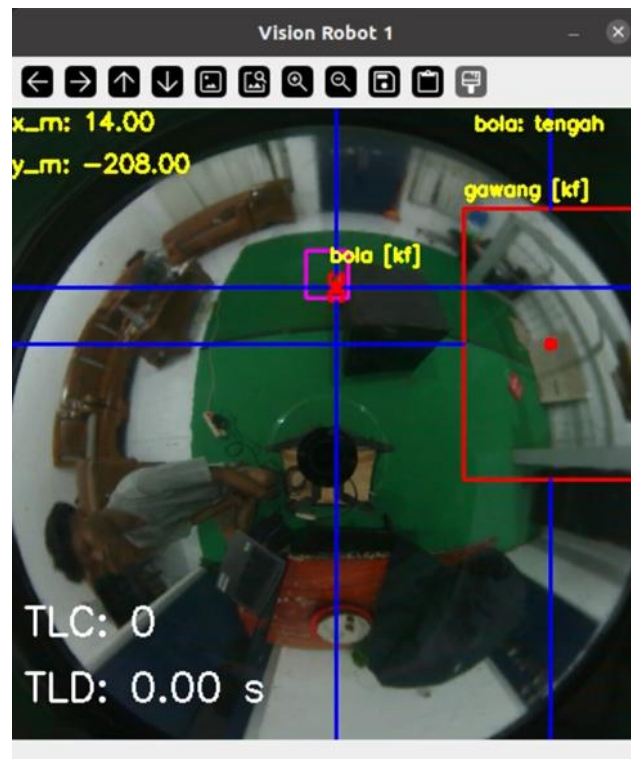


Figure 7. Optimized system maintaining tracking under 60% occlusion (TLC: 0)

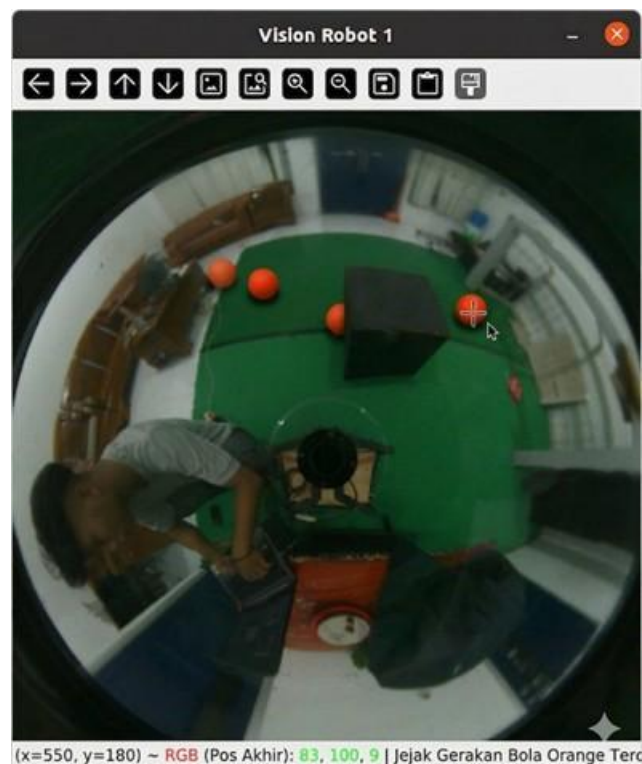


Figure 8. Illustration of the Momentary Occlusion scenario where the ball moves behind an obstacle



Figure 9. Baseline system losing track during momentary occlusion (TLD: 2.89 s)



Figure 10. Optimized system successfully predicting ball trajectory (TLD: 0.00 s)

To further challenge the system's predictive capabilities, a momentary occlusion scenario was introduced, as illustrated in Figure 7, where the ball rolls behind a solid obstacle for 1 to 2 seconds. The visual logs reveal a stark contrast. The baseline system (Figure 8) suffers a Total Loss Duration (TLD) of 2.89 seconds, leaving the robot effectively blind. In sharp contrast, the optimized system (Figure 9) maintains an uninterrupted trajectory projection, yielding a TLD of 0.00 seconds. This proves that the Kalman Filter's velocity parameters (v_x, v_y) effectively bridge the visual gap.

In Table 2, the baseline system significantly lost track of the ball during moderate and momentary occlusion. In contrast, the optimized system recorded 0 (zero) tracking loss in all scenarios.

Table 2. Comparison of Tracking Loss in Occlusion Test

Scenario	Baseline System		Optimized System	
	Loss Count	Duration (s)	Loss Count	Duration (s)
Mild Occlusion (30%)	0	0	0	0
Moderate Occlusion (60%)	96	13.94	0	0
Momentary Occlusion (1-2 s)	6	2.89	0	0

These results prove that the Kalman Filter functions as "spatial memory" for the robot. When the ball is occluded and YOLO fails to detect it, the Kalman Filter continues to project the ball's trajectory based on velocity estimation (v_x, v_y) from the previous state vector [11]. This capability is crucial for avoiding navigational confusion when the robot collides with opponents or the ball is momentarily obstructed. This finding aligns with the research by Takura et al. [17], which showed that the Kalman Filter effectively tracks occluded objects in video.

C. Motion Stability and Mission Time

The robot motion command log data presented in Table 3 shows a significant reduction in jitter.

Table 3. Comparison of Motion Command Frequency (5 Repetitions)

Command Type	Baseline System	Optimized System
turnLeft / turnLeftSlow	44 times	11 times
turnRight / turnRightSlow	30 times	12 times
forward / forwardSlow	48 times	10 times
Total Direction Correction Commands	74 times	23 times

This 69% reduction in jitter directly impacts the efficiency of mission completion time, as shown in Table 4.

Table 4. Average Mission Completion Time in Dynamic Test

Repetition	Baseline System (seconds)	Optimized System (seconds)
1	12.400	7.254
2	19.277	5.350
3	12.986	5.987
4	19.139	5.615
5	12.057	4.638
Average	15.171	5.768

The optimized system consistently achieved a faster mission completion time, reducing the average duration by 62% (from 15.171 seconds to 5.768 seconds). This significant improvement highlights the system's ability to maintain a stable and continuous trajectory without the sudden stops or interruptions typically observed in the baseline system. Robot movement became smoother and more linear because the Kalman Filter estimation eliminates coordinate fluctuations that typically arise from YOLO detection noise or momentary lighting changes.

4. CONCLUSION

Based on the research results, the integration of the Kalman Filter into the YOLOv8 tracking system has proven effective in optimizing the motion of KRSBI-Beroda robots. The specific conclusions obtained are:

1. The optimized system successfully increased the average FPS by 185% (from 13.6 to 38.77), overcoming computational bottlenecks typical of consumer-grade hardware used in the competition.
2. The Kalman Filter provided perfect resilience against occlusion (0% tracking loss), ensuring the robot does not lose direction when the ball is obstructed—a critical advantage in dynamic gameplay.
3. Motion stability significantly improved with a 69% reduction in motion command jitter (from 74 times to 23 times), resulting in 62% faster mission completion (from 15.17 seconds to 5.76 seconds).

The proposed method offers a cost-effective, software-based solution that can be readily adopted by other KRSBI teams facing similar hardware limitations. For further development, it is recommended to use edge computing devices such as NVIDIA Jetson to run YOLO on every frame and to explore the Extended Kalman Filter (EKF) to model the non-linear dynamics of the ball [18].

ACKNOWLEDGEMENT

The authors would like to express their sincere gratitude to all parties who have supported the completion of this research and the writing of this article. Special thanks are extended to the robotics team members who assisted during the data collection and experimentation phases. Their contributions, encouragement, and assistance are deeply appreciated.

REFERENCES

- [1] B. Kusumoputro *et al.*, “Pedoman Kontes Robot Indonesia (Kri) Pendidikan Tinggi Tahun 2024,” 2024.
- [2] B. M. Nanda, S. Siregar, and M. I. Sani, “Implementasi Object Detection Pada Robot Sepak Bola Beroda Berbasis Kamera Omnidirectional Menggunakan OpenCV,” vol. 9, no. 4, pp. 2064–2068, 2023.
- [3] T. M. Farhan and F. Candra, “CNN-Based Ball and Goal Detection for KRSBI Robot with Omnidirectional Camera,” *Int. J. Electr. Energy Power Syst. Eng.*, vol. 8, no. 2, pp. 86–98, 2025, doi: 10.31258/ijeepse.8.2.1-13.
- [4] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You Only Look Once: Unified, Real-time Object Detection,” *Proc. IEEE Comput. Soc. Conf. Comput. Vis. Pattern Recognit.*, vol. 2016-Decem, pp. 779–788, 2016, doi: 10.1109/CVPR.2016.91.
- [5] M. Yaseen, “What Is Yolov8: An In-Depth Exploration Of The Internal Features Of The Next-Generation Object Detector,” vol. 8, pp. 1–10, 2024.
- [6] Ultralytics, “Kalman Filter (KF).” [Online]. Available: <https://www.ultralytics.com/glossary/kalman-filter-kf>
- [7] M. Surya, A. Nehemia Toscani, C. Saputra, Y. Pratama, and M. I. Bustami, “Penggunaan Yolo Untuk Deteksi Robot Dan Gawang Pada Robot Sepak Bola Beroda,” *Indones. J. Comput. Sci.*, vol. 14, no. 1, pp. 1055–1070, 2025, doi: 10.33022/ijcs.v14i1.4575.

- [8] S. E. Ryu and K. Y. Chung, "Detection Model Of Occluded Object Based On Yolo Using Hard-example Mining And Augmentation Policy Optimization," *Appl. Sci.*, vol. 11, no. 15, 2021, doi: 10.3390/app11157093.
- [9] J. Nuralim, N. Fath, A. Musafa, Sujono, and D. S. Broto, "Perancangan Sistem Pendeteksian Obyek Bola Dengan Metode Framework YOLOv4," *J. Maest.*, vol. 5, no. 2, pp. 289–294, 2022, [Online]. Available: <https://jom.ft.budiluhur.ac.id/index.php/maestro/article/view/531>
- [10] R. Sholehurrohman, M. R. Habibi, I. S. Ilman, R. Taufiq, and M. Muhaqiqin, "Analisis Metode Kalman Filter, Particle Filter dan Correlation Filter Untuk Pelacakan Objek," *Komputika J. Sist. Komput.*, vol. 12, no. 2, pp. 21–28, 2023, doi: 10.34010/komputika.v12i2.9567.
- [11] C. Saputra, "Implementasi Algoritma SIFT (Scale-Invariant Feature Transform) Dan Algoritma Kalman Filter Dalam Mendeteksi Objek Bola," *J. Process.*, vol. 18, no. 1, pp. 73–82, 2023, doi: 10.33998/processor.2023.18.1.791.
- [12] Y. G. Lee, "Confidence-Guided Frame Skipping to Enhance Object Tracking Speed," *Sensors*, vol. 24, no. 24, 2024, doi: 10.3390/s24248120.
- [13] L. E. Taylor, M. Mirdanies, and R. P. Saputra, "Optimized Object Tracking Technique Using Kalman," vol. 07, pp. 57–66, 2016, doi: 10.14203/j.mev.2016.v7.57-66.
- [14] M. Quigley *et al.*, "ROS : An Open-source Robot Operating System," no. Figure 1, 2009.
- [15] OpenCV, "Open Source Computer Vision Library." [Online]. Available: <https://opencv.org/>
- [16] NumPy, "NumPy." [Online]. Available: <https://numpy.org/>
- [17] K. E. I. Takura, Y. U. M. A. N. Arita, and S. H. N. Oaki, "Automatic pear and apple detection by videos using deep learning and a Kalman filter," vol. 4, no. 5, pp. 1688–1695, 2021.
- [18] A. Z. Firdaus and D. Lelono, "Sistem Klasifikasi Sampah Otomatis Berbasis Deteksi Objek Real-Time Pada Single Board Computer Dengan Algoritma YOLO," vol. 15, no. 1, pp. 49–60, 2025, doi: 10.22146/ijeis.104520.